Repository Assistant — Chatbot & Search API (Frontend Guide)

The V6 API proxies the repository assistant to the Agents service (`POST /chat`). The frontend only calls V6; it never calls Agents directly.

Feature	V6 endpoint	Agents intent	Response returned to frontend
Chatbot	`POST /api/repositories/assistant/chatbot`	`chatbot`	`chatbot_result` object only
Global search	`POST /api/repositories/assistant/search`	`global_search`	`global_search_result` object only

Source: `src/Api/Controllers/RepositoryAssistantController.cs`

Authentication

- Header: `Authorization: Bearer <accessToken>` (required)
- Policy: `TenantUser` — the caller must be a logged-in tenant user.
- `tenantId` is taken from the logged-in user's token when you don't send it.

---

1. Chatbot

```
POST {baseUrl}/api/repositories/assistant/chatbot
Content-Type: application/json
Authorization: Bearer <accessToken>
```

Request body

Field	Type	Required	Notes
`message`	string	**Yes**	The user's chat message. Empty → `400`.
`sessionId`	string	No	Conversation id. **Send the same value on every message in one chat** so the agent keeps context. If omitted, V6 generates a new `repo-assistant-<guid>` each call (no memory between messages).
`specificId`	guid	No	Limit the chat to one repository (repository id).
`actionFrom`	string	No	Defaults to `"Repository"`.
`tenantId`	guid	No	Defaults to the tenant from the token.
`token`	string	No	Not needed. When omitted, V6 forwards the `Authorization` header value to the agent.

Example

```
{
  "message": "show invoices from vendor 3101",
  "sessionId": "console-7f3c2a9e-1b4d-4c1a-9e2f-123456789abc",
  "specificId": "0b9d6a3e-2f1c-4e8a-9a51-6c1f2d3e4b5a"
}
```

Generating `sessionId` (frontend)

Create one when the chat panel opens (or when the user clicks "New chat") and keep it until the chat is closed:

```
const sessionId = `console-${crypto.randomUUID()}`;
```

What V6 sends to Agents (for reference)

```
{
  "session_id": "console-7f3c2a9e-...",
  "intent": "chatbot",
  "message": "show invoices from vendor 3101",
  "payload": {
    "query": "show invoices from vendor 3101",
    "tenantId": "…",
    "specificId": "…",
    "actionFrom": "Repository",
    "token": "Bearer eyJ…"
  }
}
```

Response

On `2xx`, V6 unwraps the Agents envelope and returns **only** the `chatbot_result` object as `application/json`. Its fields are defined by the Agents service (not by V6), so render it according to the

agent's contract.

If the agent's reply has no `chatbot_result` (or isn't JSON), V6 returns the agent's raw body unchanged.

---

2. Global search

```
POST {baseUrl}/api/repositories/assistant/search
Content-Type: application/json
Authorization: Bearer <accessToken>
```

Request body

Field	Type	Required	Notes
`query`	string	**Yes**	Search text. Empty → `400`.
`specificId`	guid	No	Limit search to one repository.
`actionFrom`	string	No	Defaults to `"Repository"`.
`tenantId`	guid	No	Defaults to the tenant from the token.

```
{ "actionFrom": "Repository", "query": "po", "specificId": "0b9d6a3e-…" }
```

Search has no session — each call is independent.

Response

On `2xx`, returns **only** the agent's `global_search_result` object.

Errors (both endpoints)

Status	When	Body
`400`	`message` / `query` missing or blank	`{ "error": "message is required." }` / `{ "error": "query is required." }`
`401` / `403`	Missing/invalid token, or not a tenant user	—
`502`	Assistant disabled, `Agents:ChatUrl` not configured, or Agents unreachable	`{ "error": "Repository Python chatbot service is unavailable: …" }`
Other non-2xx	Passed through from Agents unchanged	Agent's raw body

Requests can take a while — the server-side timeout is **120 s**, so set the frontend HTTP timeout at or above that and show a "thinking…" indicator.

Server configuration (backend / DevOps)

```
"Agents": {
  "ChatUrl": "https://cloud.ezofis.com/chat"
},
"RepositoryPythonAssistant": {
  "Enabled": true,
  "ChatUrl": "",
  "SearchIntent": "global_search",
  "ChatbotIntent": "chatbot",
  "TimeoutSeconds": 120
}
```

- `RepositoryPythonAssistant:ChatUrl` overrides `Agents:ChatUrl` when set.
- Production (docker) uses `http://agents:8000/chat`.
- `Enabled: false` makes both endpoints return `502`.

Frontend integration checklist

1. Open chat → generate `sessionId`.
2. On send → `POST /api/repositories/assistant/chatbot` with `{ message, sessionId, specificId? }`.
3. Show a loading state (up to ~120 s).
4. Render the returned `chatbot_result` object.
5. `400` → validation message; `502` → "Assistant is unavailable, try again".
6. "New chat" → generate a new `sessionId`.